

Implémentation, ServeurMultiThreade (v4)

Fabrice.Kordon@lip6.fr



Objectif de cette version

2

Contrôler le parallélisme

-  Avec un pool de threads pour les servants



Peu de changements...

3

```
import java.util.Random;  
import java.util.concurrent.ArrayBlockingQueue;  
import java.util.concurrent.ExecutorService; // nouvelle dépendance  
import java.util.concurrent.Executors; // nouvelle dépendance
```

```
class Requete {...}  
class Servant implements Runnable {...}  
class UnClient implements Runnable {...}  
public class ServeurMultiThreade4 {...}
```


Peu de changements...

3

```
import java.util.Random;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.ExecutorService; // nouvelle dépendance
import java.util.concurrent.Executors; // nouvelle dépendance
```

```
class Requete {...}
class Servant implements Runnable {...}
class UnClient implements Runnable {...}
```

A terminal window with a dark background and yellow text. The window title bar shows standard macOS window controls (red, yellow, green buttons) and a maximize button. The terminal content shows a diff command being executed, comparing two Java files. The output shows several changes, including the addition of imports for ExecutorService and Executors, and the addition of a new class ServantMultiThreade4.

```
$ diff ServeurMultiThreade3.java ServeurMultiThreade4.java
2d1
< //import java.util.concurrent.locks.*;
3a3,4
> import java.util.concurrent.ExecutorService; // nouvelle dépendance
> import java.util.concurrent.Executors; // nouvelle dépendance
45a47,49
> // Borner le parallélisme des servants à 3
> private final int MAX_SERV = 3;
> private ExecutorService executor = Executors.newFixedThreadPool(MAX_SERV);
63c67
<         new Thread (new Servant(r)).start();
---
>         executor.execute (new Servant(r));
67a72
>         executor.shutdown();
133c138
< public class ServeurMultiThreade3 {
---
> public class ServeurMultiThreade4 {
```


La classe LeServeur

4

```
class LeServeur implements Runnable {
    private ArrayBlockingQueue<Requete> f;
    // Borner le parallélisme des servants à 3
    private final int MAX_SERV = 3;
    private ExecutorService executor = Executors.newFixedThreadPool(MAX_SERV);

    LeServeur (ArrayBlockingQueue<Requete> fr) {...}
    private void trace (String m) {...}

    public void run() {
        try {
            Requete r;
            this.trace("initialisé");
            while (true) {
                r = f.take(); // potentiellement bloquant
                this.trace("je traite " + r);
                executor.execute (new Servant(r)); // Dans le pool
            }
        } catch (Exception e) { // Gérer la terminaison
            this.trace("il n'y a plus personne, adieu monde cruel");
            executor.shutdown(); // Terminer le pool
        }
    }
}
```


Une exécution

5

```
$ java ServeurMultiThreade4 2 4
UnClient 1 : initialisé
UnClient 4 : initialisé
UnClient 3 : initialisé
UnClient 2 : initialisé
Serveur : initialisé
UnClient 1 : j'envoie la requête (client 1, 921)
UnClient 3 : j'envoie la requête (client 3, 567)
UnClient 2 : j'envoie la requête (client 2, 869)
UnClient 4 : j'envoie la requête (client 4, 447)
UnClient 1 : j'attends le serveur
Serveur : je traite (client 1, 921)
UnClient 2 : j'attends le serveur
UnClient 3 : j'attends le serveur
Serveur : je traite (client 3, 567)
UnClient 4 : j'attends le serveur
Servant (client 1, 921) : initialisé
Serveur : je traite (client 2, 869)
Servant (client 3, 567) : initialisé
Serveur : je traite (client 4, 447)
Servant (client 2, 869) : initialisé
Servant (client 3, 567) : je prévient le client
UnClient 3 : ma requête a été traitée
Servant (client 4, 447) : initialisé
Servant (client 2, 869) : je prévient le client
UnClient 2 : ma requête a été traitée
Servant (client 1, 921) : je prévient le client
UnClient 1 : ma requête a été traitée
Servant (client 1, 921) : je prévient le client
UnClient 1 : ma requête a été traitée
Servant (client 4, 447) : je prévient le client
UnClient 4 : ma requête a été traitée
UnClient 4 : étant le dernier, je tue le serveur
Serveur : il n'y a plus personne, adieu monde cruel
```


Une exécution

5

```
$ java ServeurMultiThreade4 2 4
UnClient 1 : initialisé
UnClient 4 : initialisé
UnClient 3 : initialisé
UnClient 2 : initialisé
Serveur : initialisé
UnClient 1 : j'envoie la requête (client 1, 921)
UnClient 3 : j'envoie la requête (client 3, 567)
UnClient 2 : j'envoie la requête (client 2, 869)
UnClient 4 : j'envoie la requête (client 4, 447)
UnClient 1 : j'attends le serveur
Serveur : je traite (client 1, 921)
UnClient 2 : j'attends le serveur
UnClient 3 : j'attends le serveur
Serveur : je traite (client 3, 567)
UnClient 4 : j'attends le serveur
Servant (client 1, 921) : je préviens le client
Serveur : je traite (client 2, 869)
Servant (client 3, 567) : je préviens le client
Serveur : je traite (client 4, 447)
UnClient 3 : ma requête a été traitée
Servant (client 4, 447) : je préviens le client
UnClient 2 : ma requête a été traitée
Servant (client 1, 921) : je préviens le client
UnClient 1 : ma requête a été traitée
Servant (client 1, 921) : je préviens le client
UnClient 1 : ma requête a été traitée
```

```
Servant (client 4, 447) : je préviens le client
UnClient 4 : ma requête a été traitée
UnClient 4 : étant le dernier, je tue le serveur
Serveur : il n'y a plus personne, adieu monde cruel
```



L'exécution reste identique

Mais le parallélisme max change
Quel est-il?



**3 (pool) + 1 (serveur) +
N (clients) + 1 (main)**

Mettre les client en pool?